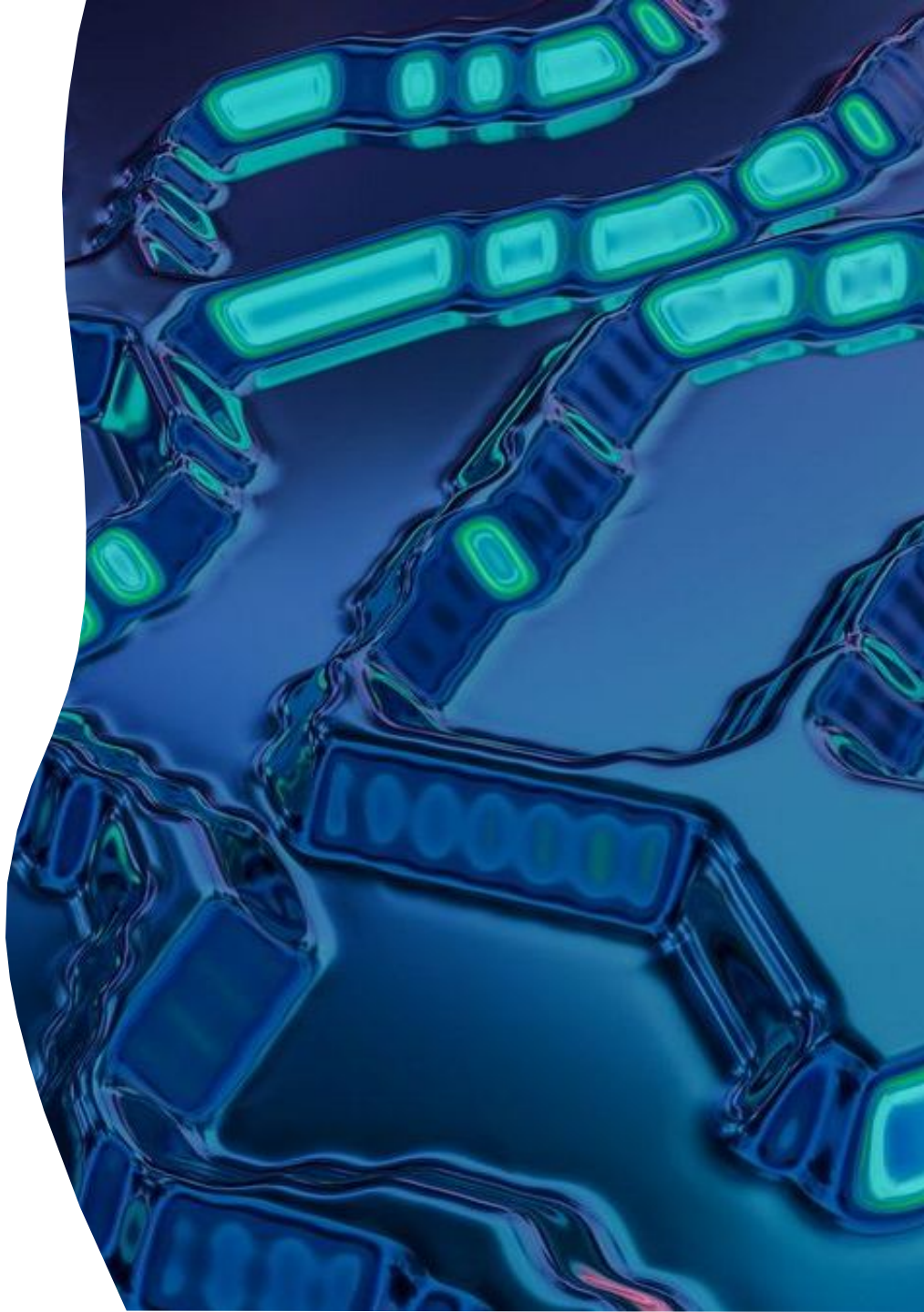
Data Analytics, Data Mining and Machine Learning

Lecture 8

Narges Chinichian · Semester 1, 2026



Overview of this course

Week	Lecture Num	Date	Topic
1	Lecture 1	07.09.2026	Introduction
2	Lecture 2	14.09.2026	Data, Research Questions & Ethics
3	Lecture 3	21.09.2026	Data Visualisation & Exploratory Data Analysis
4	Lecture 4	28.09.2026	Preprocessing & Feature Engineering
5	Lecture 5	05.10.2026	Machine Learning Fundamentals & Regression
6	—	12.10.2026	Project Presentations I
7	Lecture 6	19.10.2026	Classification: Logistic Regression, KNN [Decision Trees is moved to the next session]
8	—	26.10.2026	Bank Holiday — No Class
9	Lecture 7	02.11.2026	Gradient Descent, Regularisation & Model Complexity [+ Decision Trees]
10	Lecture 8	09.11.2026	Model Evaluation & Ensemble Methods
11	Lecture 9	16.11.2026	Neural Networks, Unsupervised Learning & Special Topics
12	—	23.11.2026	Project Presentations II

Quick Quiz

Check Loop





Recap: Three Classification Models

Logistic Regression

models class probability; learns a linear decision boundary

KNN

classifies by majority class among the k nearest observations

Decision Trees

learn sequential if–then splitting rules

Key question: How do we know which model performs best?



Why Model Evaluation Matters

- Training performance does not guarantee good predictions on unseen data
- We want models that generalise, not memorise
- Evaluation must be fair and independent of model fitting

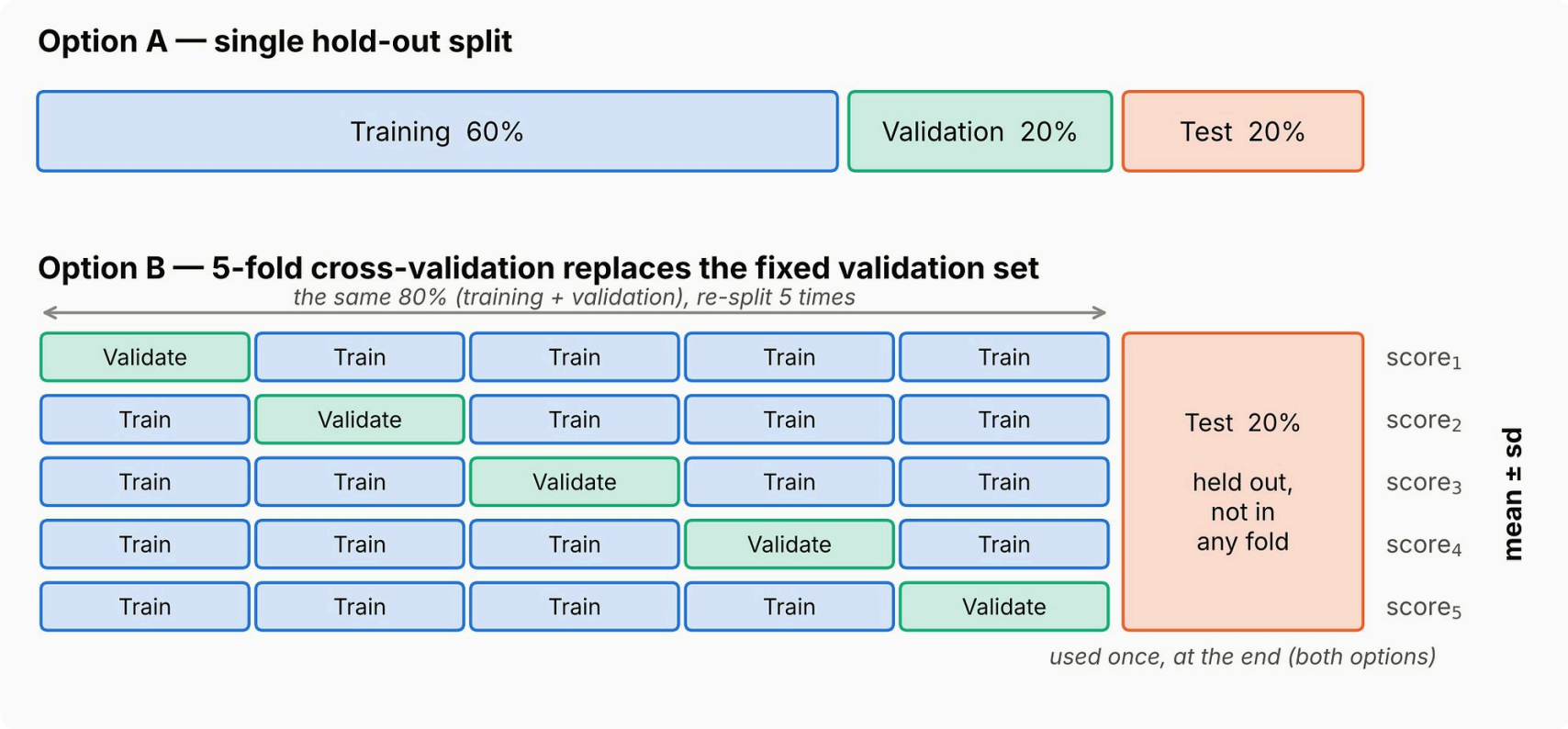
Train, Validation & Test Sets

Training set
fit model parameters

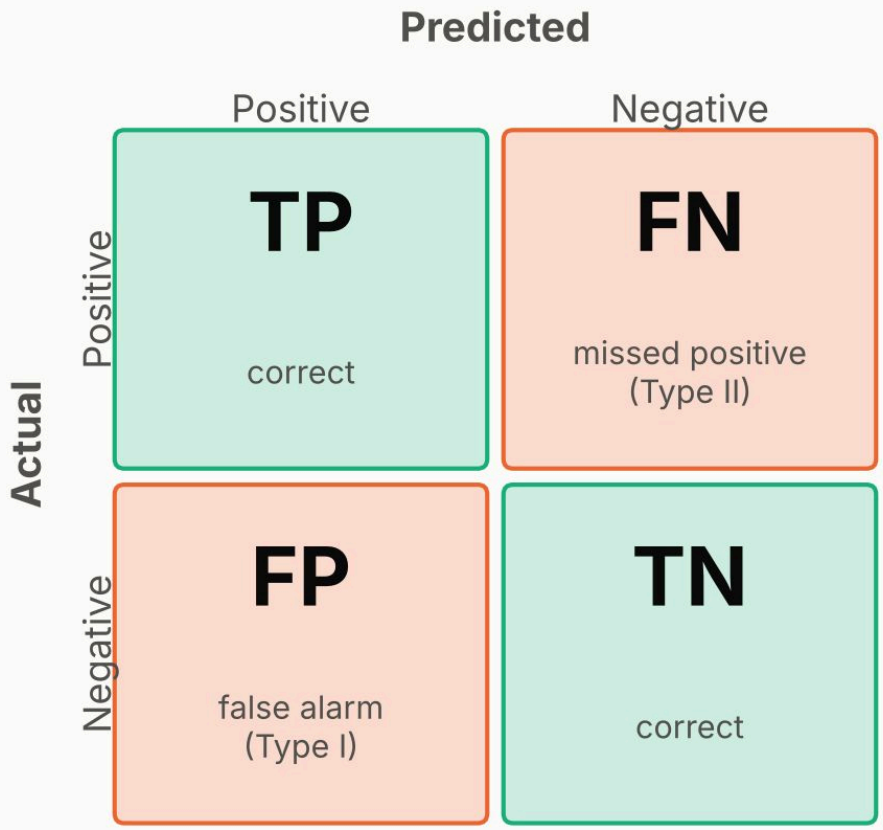
Validation set
select models and tune hyperparameters

Test set
estimate final performance, used once

⚠ **Data leakage:** information unavailable at prediction time, or drawn from evaluation data, enters training and inflates performance
Example: fitting a scaler on the full dataset before splitting



The Confusion Matrix



Accuracy $\frac{TP + TN}{TP + TN + FP + FN}$

Precision $\frac{TP}{TP + FP}$

Recall $\frac{TP}{TP + FN}$

F1 $\frac{2 \cdot P \cdot R}{P + R}$

⚠ Accuracy can be misleading with imbalanced classes

Comparing Models Fairly

- Compare models on the same validation data or cross-validation folds
- **k-fold cross-validation:** each observation is used for validation exactly once
- **Hyperparameter tuning:** use validation data or cross-validation, never the test set
- Selecting a model on test results biases the estimate upward
- Touch the test set once, for final evaluation

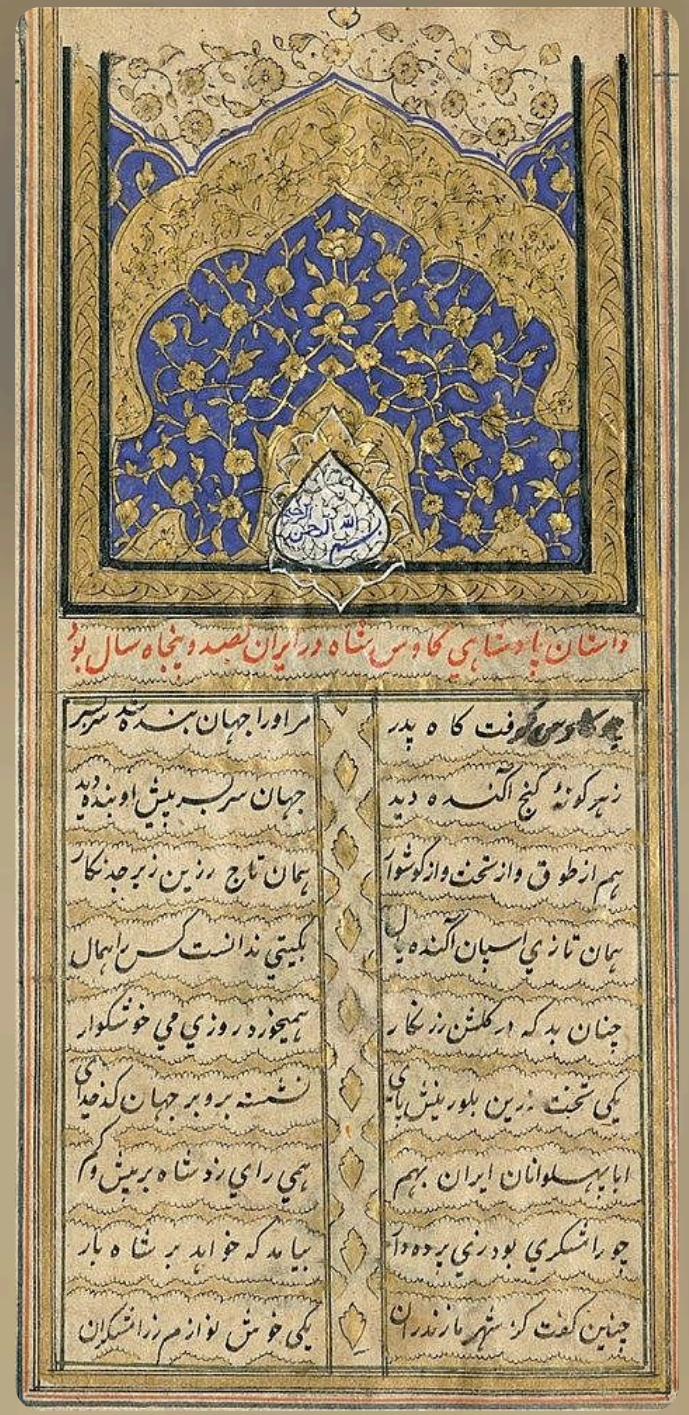
From Individual Models to Collective Intelligence

«همه چیز را همگان دانند...»

"All things are known by all people..." — Persian proverb, attributed to Bozorgmehr (6th century CE) from Qābus-nāma — 1082 CE

- No single model knows everything.
- This is the central idea behind ensemble learning

Image credit: **invaluable**



Three Approaches to Ensemble Learning

Ensemble methods enhance predictive performance by strategically combining multiple individual models.

1

Bagging (Bootsprap aggregating)

2

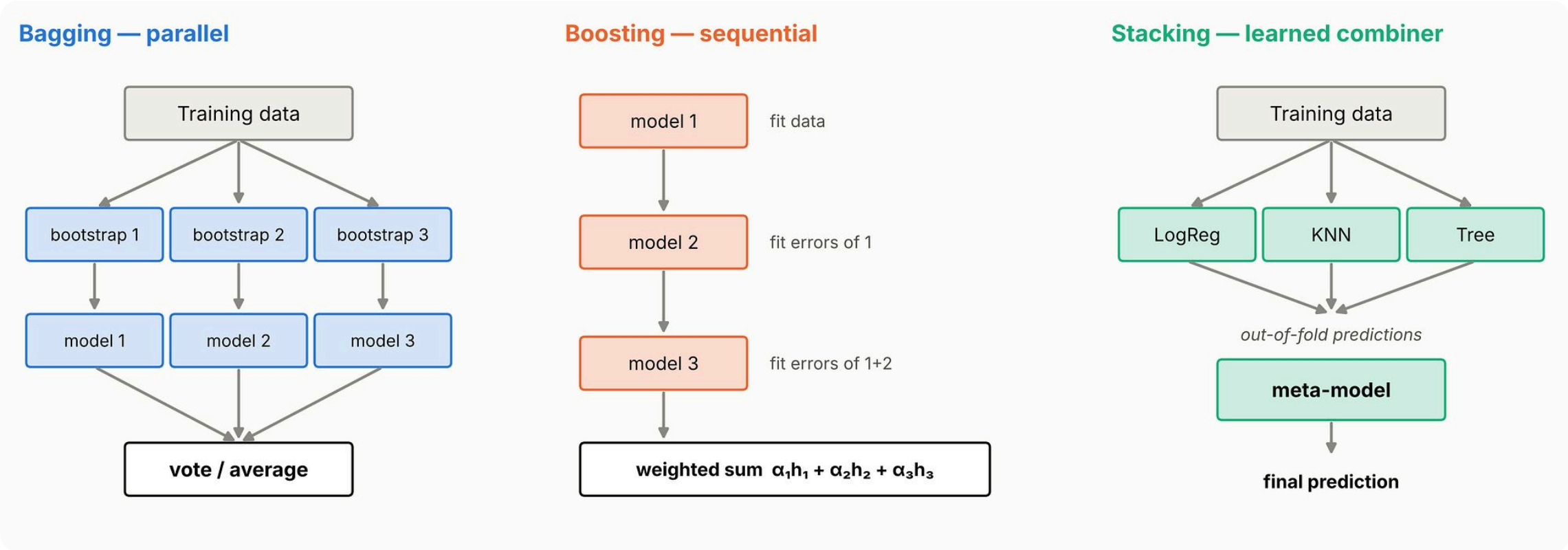
Boosting

3

Stacking

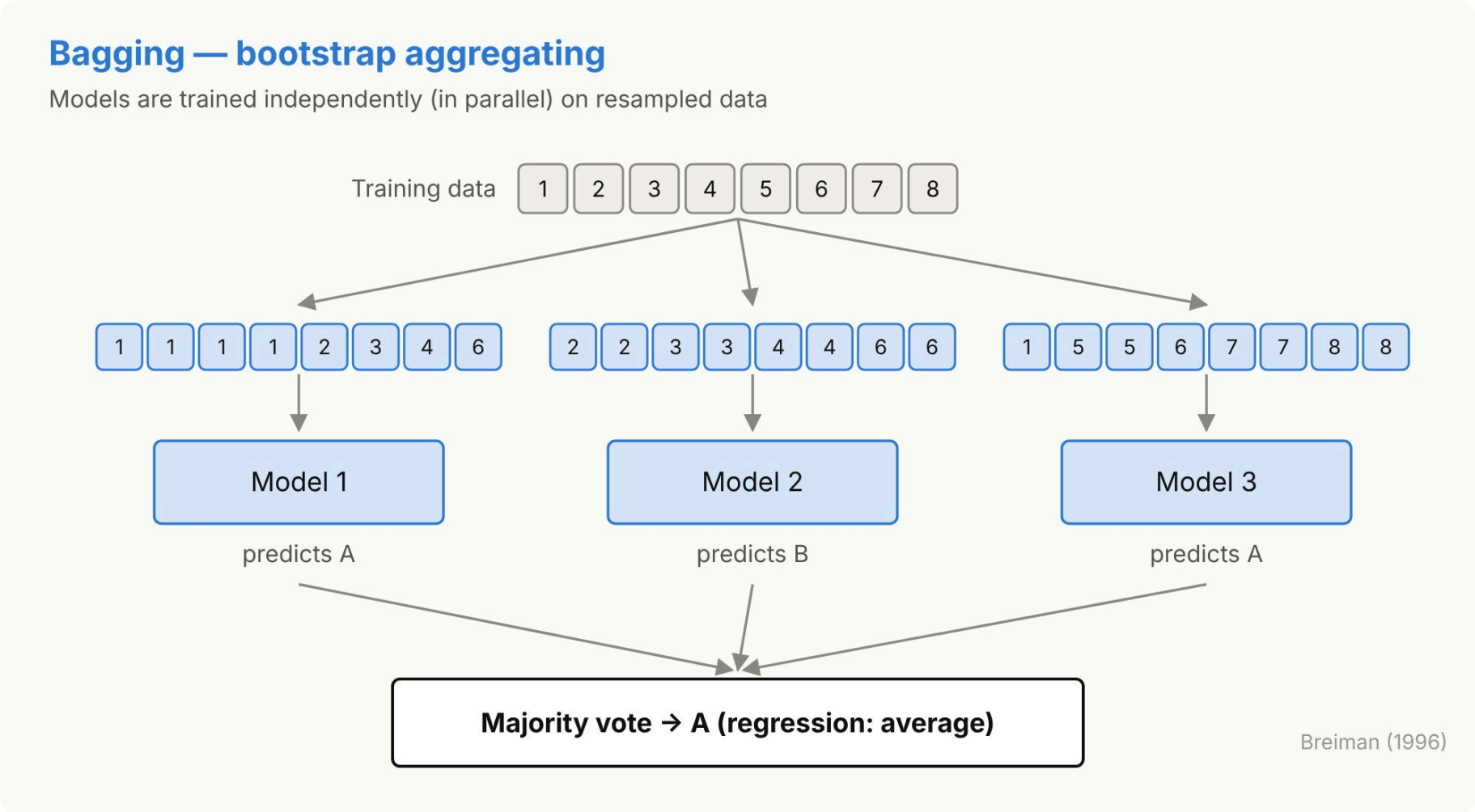
Generally, **voting** mechanisms combine class predictions, while **averaging** is used for numerical values or probabilities.

Three Approaches to Ensemble Learning



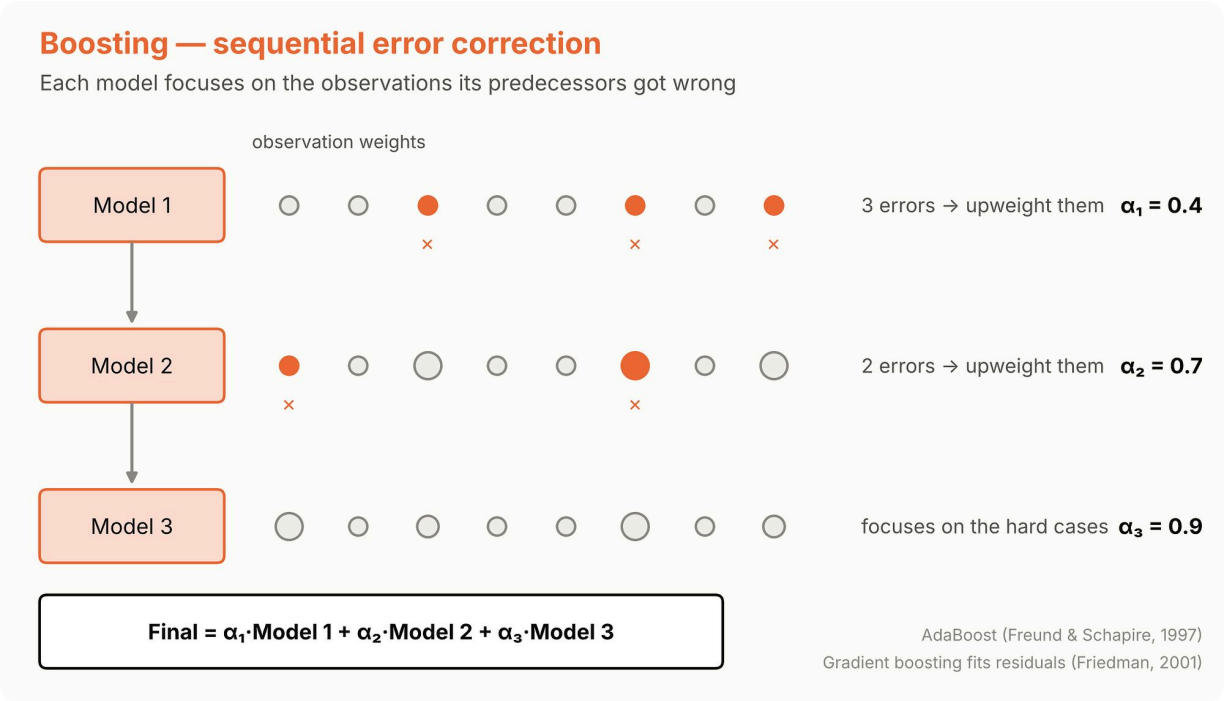
Bootstrap Aggregating: Models are trained independently on different bootstrap [same size, drawn with replacement] samples of the training data. Their predictions are then combined through **majority voting** (for classification) or **averaging** (for regression). This reduces variance and overfitting.

(Breiman, 1996)

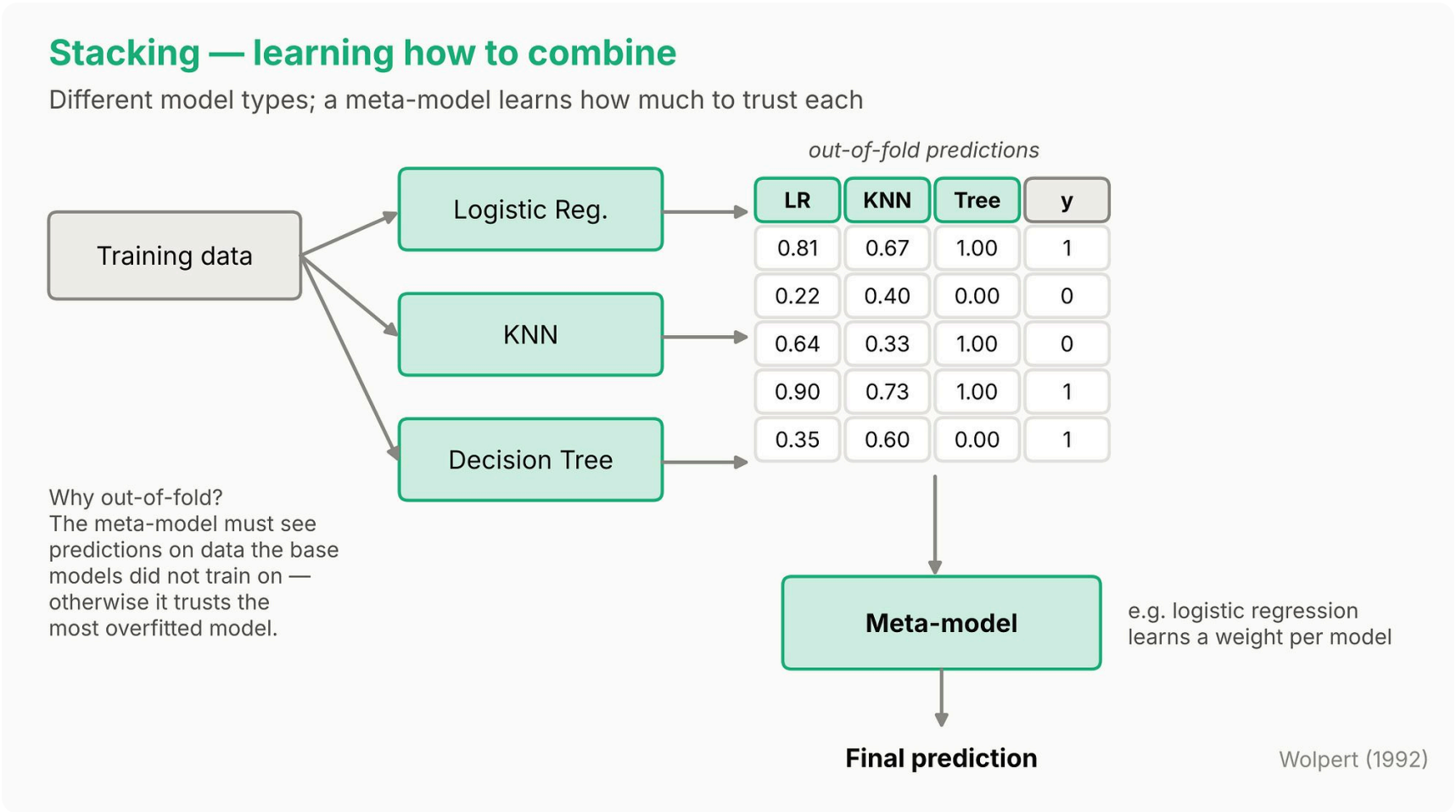


Sequential Learning: Models are trained sequentially, with each new model focusing on correcting the errors of its predecessors. Final predictions are typically a **weighted sum** of all models. This reduces bias.

- AdaBoost reweights misclassified observations (Freund & Schapire, 1997)
- Gradient boosting fits the residual errors (Friedman, 2001)



Meta-Learning: Diverse models (base learners) are trained, and their predictions are used as input to a higher-level model (meta-learner). The meta-learner learns the optimal way to combine the predictions of the base models, often trained on **out-of-fold predictions** to prevent data leakage. (Wolpert, 1992)



Note: in **Out-of-fold predictions** each training row's prediction from a model trained on the other folds, used as the meta-model's input instead of being averaged into a score.

Bagging vs. Boosting vs. Stacking in one place:

	Bagging	Boosting	Stacking
In one line	Copies of one model on resampled data, then vote	Models in sequence, each correcting the previous errors	Different models, combined by a second model
Base models	**Same algorithm**, same settings	**Same algorithm**, same settings	**Different algorithms**
Best suited base model	Unstable: low bias, high variance (e.g. deep decision trees, neural networks)	Weak: high bias, only slightly better than chance (e.g. shallow trees, decision stumps)	Strong and different from each other (e.g. random forest, logistic regression, support vector machine, k-nearest neighbours)
Training data	Each model gets its own **bootstrap sample**: same size, drawn with replacement	Full data each round, but focused on errors: **rows reweighted** (AdaBoost) or the model **fits the remaining errors** (gradient boosting)	Base models: full data. Combiner: the base models' **out-of-fold predictions** from cross-validation
Training order	**Parallel**: models are independent	**Sequential**: each model depends on the previous ones	Base models in parallel, then the combiner
Combining predictions	Majority vote (classification) or average (regression), **equal weights**	**Weighted** sum: by each model's accuracy (AdaBoost) or scaled by a learning rate (gradient boosting)	A **meta-model** learns how to weight the predictions, often logistic or linear regression
Main risk	Little gain with stable models such as linear regression; harder to interpret than one model	Overfitting with too many rounds; sensitive to noisy labels and outliers; needs tuning	**Data leakage** if the meta-model is trained on in-sample predictions; slow and complex
Well-known examples	Random forest (bagging plus random feature subsets), bagged neural networks	AdaBoost, gradient boosting, XGBoost, LightGBM, CatBoost	scikit-learn `StackingClassifier`, competition-winning ensembles

Let's look at one famous ensemble model:

Random Forest: Many Trees, One Prediction

- Each tree is trained on a bootstrap sample
- Each split considers only a random subset of features
- **Classification:** majority vote (Breiman, 2001); scikit-learn averages class probabilities
- **Regression:** average of tree predictions
- Reduces the high variance (instability) of individual deep trees

Key Takeaways

- Evaluation metrics should reflect the problem
- Validation and cross-validation guide model selection; the test set gives the final estimate
- Ensembles combine models by voting, averaging, additive boosting or meta-learning
- Random Forest shows how combining decorrelated trees reduces variance

Hands-On:
Evaluating Models and Ensembles Models

